

ParcialT1yT2Nuria.pdf



Yapper



Programación y Diseño Orientado a Objetos



2º Grado en Ingeniería Informática



**Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada**



MÁSTER EN

Inteligencia Artificial & Data Management

MADRID

Formamos
talento para un futuro
Sostenible

saber más



CAMBIA TU VIDA ESTE VERANO



EXAMEN PARCIAL DE PDOO (Temas 1 y 2)

Nombre: _____

Este examen se acompaña de un diagrama de clases y un diagrama de secuencia.

1) Preguntas sobre conceptos de programación y diseño orientado a objetos. (2 puntos)

Responde verdadero (V) o falso (F). Cada respuesta incorrecta anula una respuesta correcta.

UML es un lenguaje de modelado unificado que puede ser usado para diseñar programas orientados a objetos con independencia del lenguaje, y que permite especificar (entre otras cosas) la relación estructural entre las clases del programa y la interacción de los objetos durante la ejecución del mismo.	
Un atributo de instancia puede ser usado dentro de un método de clase.	
En Ruby todas las clases son también objetos.	
Dos objetos con la misma identidad pueden estar almacenados en distintas partes de la memoria dinámica.	
Dos objetos diferentes de una misma clase no pueden tener el mismo estado.	
Cada objeto guarda su propia copia de los atributos de instancia y de clase.	
En Ruby se puede cambiar la visibilidad de atributos y métodos.	
En Java, el siguiente código crea un objeto de la clase Vehiculo sin inicializar: Vehiculo v;	
Los módulos de Ruby pueden contener trozos de código que no estén dentro de una clase.	
En Java y en Ruby existe un recolector automático de basura por lo que no hay que destruir explícitamente los objetos que ya no se usan.	

2) Observando el **diagrama de clases** (2,5 puntos):

* Asume que, aunque no aparezcan explícitamente en el diagrama de clases, todos los get y set básicos de los atributos, así como los constructores necesarios, existen para todas las clases.

2.a) Implementa la clase Trayecto en Java. Declara todos los atributos y todos los métodos (salvo gets, sets y constructores). No escribas el cuerpo de ningún método.

Ten en cuenta que:

- * La navegabilidad de Etapa a Trayecto es bidireccional.
- * La visibilidad del atributo numeroTrayectoEnEtapa es ~.
- * p es de tipo Peregrino, fechaFin es de tipo Date y horaFin es de tipo Time.
- * Las clases Time y Date se encuentran en el paquete java.util.
- * Si no se indica tipo de retorno el método no devuelve nada.

2.b) Implementa la clase Peregrino en Ruby. Debes implementar el método initialize con solo dos argumentos. El resto de métodos decláralos, pero deja el cuerpo vacío.

3) Observando el **diagrama de secuencia** (2,5 puntos):

* Asume que, aunque no aparezcan explícitamente en el diagrama de clases, todos los get y set básicos de los atributos, así como los constructores, están implementados para todas las clases.

3.a) Implementa el método 1.2, es decir: completaTrayecto de la clase Trayecto, en Java.

* Ten en cuenta los atributos que declaraste en el ejercicio 2.a) para dicha clase y usa los mismos nombres cuando sea necesario.

3.b) Implementa el método 1.2.5, es decir: etapaCompletada de la clase Etapa, en Ruby.

* Cuando sea necesario, usa Time.now.day y Time.now.hour para obtener hoy() y ahora() respectivamente.

4) Piensa cómo sería e implementa el método buscaCamina (p: Peregrino):Camina de la clase Trayecto en Java (1 punto).

WUOLAH

5) Responde V o F. Cuando F indica lo que hay que modificar para que el enunciado se haga V (2 puntos):

Enunciado	V o F	¿Cómo se haría verdadero?
Dado el siguiente código: <pre>class Bicicleta @manual_ciclismo def initialize(numero_marchas, un_color) @marchas = numero_marchas @color = un_color end end</pre> Un objeto de la clase Bicicleta tiene tres atributos de instancia.		
<pre>class MuertoViviente attr_accessor :dedos_de_frente def initialize(dedos, unaEdad) @dedos_de_frente=dedos @edad=unaEdad end def self.crearZombi(dedos) @dedos_de_frente=dedos @edad=41 end end</pre> Dado el código anterior se pueden crear objetos de esta forma: m = MuertoViviente.crearZombi(4)		
Un objeto de la clase Camino tiene un atributo de tipo SGCS.		
Dentro de la clase Trayecto se podría añadir este método y no daría error (mirar diagrama de clases): <pre>def metodo self.buscaCamina(un_peregrino) end</pre>		
<pre>class MuertoViviente def initialize() @dedos_de_frente=4.5 end def initialize(d) @dedos_de_frente=d end end</pre> Dado el código anterior se pueden crear objetos de esta forma: m = MuertoViviente.new		
<pre>import facturacion.Factura; package gallego; public class SGCS{ //código correcto }</pre> El código anterior se corresponde con el diagrama de clases y es correcto.		
Según el diagrama de clases, una Etapa no puede existir si no está ligada a un Camino.		
En el diagrama de comunicación equivalente al diagrama de secuencia proporcionado, el canal de comunicación (1.2.4) entre el objeto [t:Trayecto] y el objeto [e:Etapa] estaría estereotipado como <<A>>		
<pre>A var1 = new A("hola"); A var2 = new A("hola"); A var3 = var2;</pre> var1 y var3 son iguales e idénticos.		
En el diagrama de secuencia no existe ningún multiobjeto.		